BCA
III
SEMESTER

COMPUTER GRAPHICS

BCA
III SEMESTER

THE LEARNING HUB
INSPIRE · LEARN · LEAD
COMPUTER CENTRE
LET'S GO

1962

ACCORDING TO
LATEST
SYLLABUS

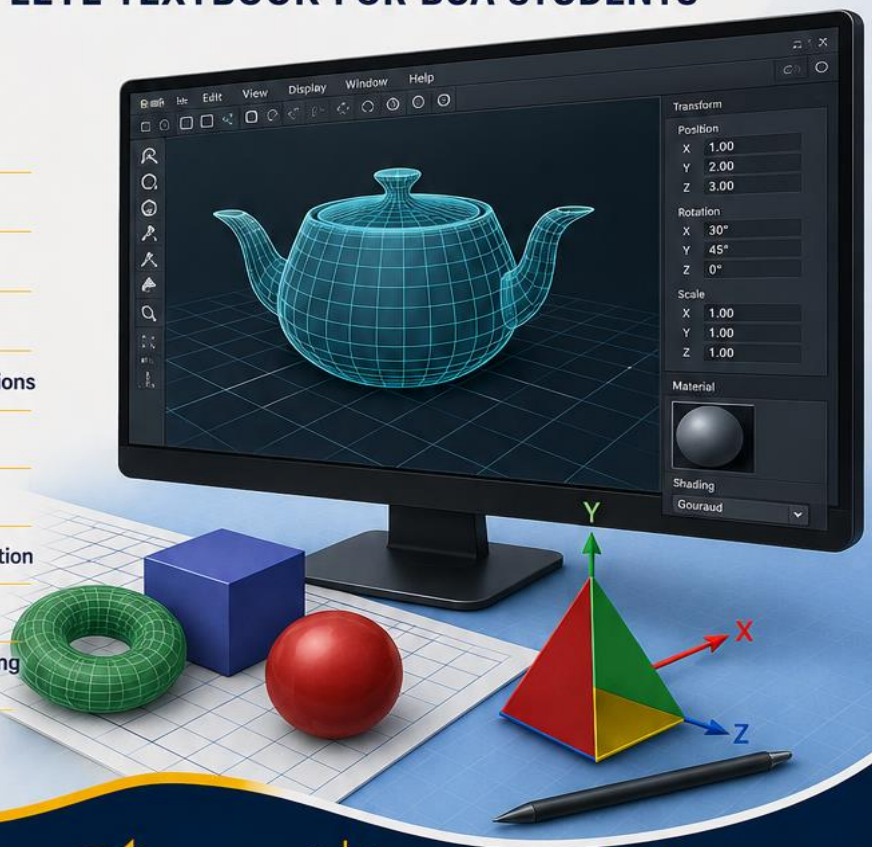
JAI NARAYAN VYAS
UNIVERSITY, JODHPUR

COMPUTER GRAPHICS

A COMPLETE TEXTBOOK FOR BCA STUDENTS

COVERS

- Introduction to Computer Graphics
- Output Primitives
- 2D Transformations
- 3D Transformations
- Viewing Transformations
- Clipping
- Curves and Surfaces
- Shading and Illumination
- Animation
- Graphics Programming Using OpenGL
- Practical Programs & Case Studies



EASY EXPLANATION

CONCEPTS WITH EXAMPLES

EXAM FOCUSED

DIAGRAMS & TABLES FOR BETTER UNDERSTANDING

LEARN TODAY, LEAD TOMORROW

UNIT - I: Computer Graphics का परिचय

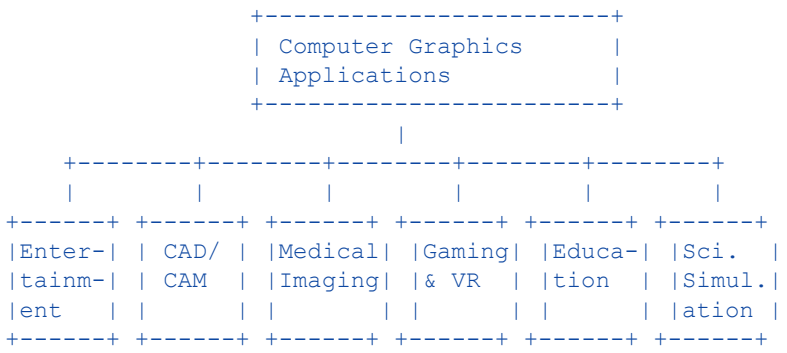
1.1 Computer Graphics क्या है?

Computer Graphics कम्प्यूटर की सहायता से visual content (चित्र, छवि, animation) बनाने, display करने और manipulate करने की तकनीक है। यह pixels, lines, curves और 3D objects को screen पर represent करती है।

1.2 Computer Graphics के मूल तत्व (Basic Elements)

तत्व (Element)	विवरण	उदाहरण
Pixel (पिक्सल)	Screen का सबसे छोटा unit	1920x1080 resolution
Point (बिंदु)	एक coordinate (x,y)	P(5,3)
Line (रेखा)	दो points को जोड़ने वाली	Drawing algorithms
Polygon (बहुभुज)	Multiple lines से बना	Triangle, Rectangle
Curve (वक्र)	Non-linear shapes	Circle, Ellipse, Bezier
Color (रंग)	RGB/HSV color model	Red(255,0,0)

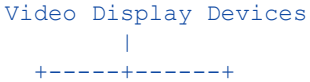
1.3 Computer Graphics के उपयोग (Applications)

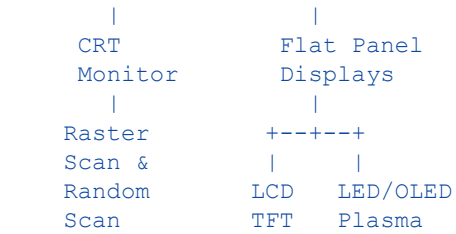


Application	विवरण
Entertainment (मनोरंजन)	Movies, Animation (Avengers, Frozen)
CAD/CAM	Engineering designs, Architecture
Medical Imaging (चिकित्सा)	MRI, CT Scan, X-Ray visualization
Gaming & VR	3D games, Virtual Reality
Education (शिक्षा)	Simulations, e-Learning
Scientific Visualization	Weather maps, Space exploration
Geographic Info Systems	Maps, Google Earth

1.4 Graphics Hardware

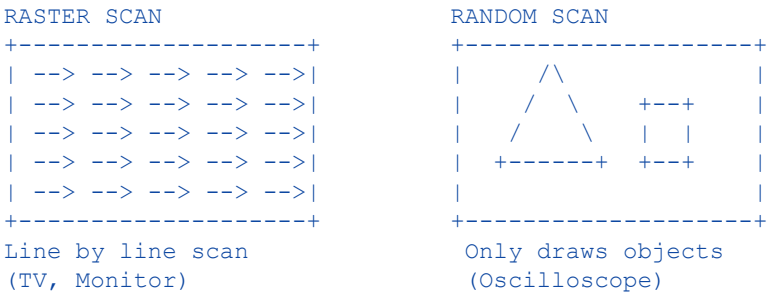
Video Display Devices (वीडियो डिस्प्ले डिवाइस):





Device	Full Form	विशेषता
CRT	Cathode Ray Tube	पुरानी technology, भारी, electron gun
LCD	Liquid Crystal Display	पतला, कम बिजली, backlit
LED	Light Emitting Diode	चमकदार, energy efficient
OLED	Organic LED	बेहतरीन contrast, flexible screens
Plasma	Plasma Display	बड़े size, high brightness

1.5 Raster Scan vs Random Scan Display



विशेषता	Raster Scan	Random Scan
Scanning	पूरी screen line by line	केवल objects को
Image Quality	Pixel-based (aliasing)	Smooth curves
Memory	Frame buffer जरूरी	Display list
Cost	कम	अधिक
Usage	TV, Computer Monitor	CAD, Vector displays
Refresh Rate	60-120 Hz	जरूरत के अनुसार

1.6 Input Devices (इनपुट डिवाइस)

Device	प्रकार	उपयोग
Mouse	Pointing device	Screen पर cursor move
Trackball	Pointing device	CAD systems
Light Pen	Pointing device	Direct screen drawing
Digitizer/Tablet	Coordinate input	Graphic design
Scanner	Image input	Photos/Documents digitize
Touch Screen	Direct input	Mobile, Kiosk

Joystick	Game input	Gaming, simulation

1.7 Hard-Copy Devices

Device	Technology	उपयोग
Inkjet Printer	Ink droplets	Photos, Documents
Laser Printer	Laser + toner	High quality documents
Plotter	Pen movement	Engineering drawings
Thermal Printer	Heat sensitive paper	Receipt printing

1.8 Line Drawing Algorithms (रेखा खींचने के एल्गोरिदम)

Line Drawing Algorithms का उपयोग screen पर दो points के बीच एक सीधी रेखा draw करने के लिए किया जाता है। हम integer coordinates (pixels) पर work करते हैं।

1.8.1 DDA Algorithm (Digital Differential Analyzer)

DDA Algorithm line drawing का सबसे सरल algorithm है। यह floating point arithmetic का उपयोग करता है।

```
Line from P1(x1,y1) to P2(x2,y2)

Steps:
1. dx = x2 - x1,  dy = y2 - y1
2. steps = max(|dx|, |dy|)
3. Xinc = dx / steps
4. Yinc = dy / steps
5. Plot (round(x), round(y)) for each step
```

```
// DDA Line Drawing Algorithm (C-style pseudocode)
void DDA(int x1, int y1, int x2, int y2) {
    int dx = x2 - x1;
    int dy = y2 - y1;
    int steps = max(abs(dx), abs(dy));

    float Xinc = (float)dx / steps;
    float Yinc = (float)dy / steps;

    float x = x1, y = y1;
    for (int i = 0; i <= steps; i++) {
        putpixel(round(x), round(y), WHITE);  // pixel plot
        x += Xinc;
        y += Yinc;
    }
}

// Example: Line from (2,3) to (6,7)
// dx=4, dy=4, steps=4
// Xinc=1.0, Yinc=1.0
// Points: (2,3)(3,4)(4,5)(5,6)(6,7)
```

विशेषता	DDA Algorithm
Advantage	Simple, Easy to implement

Disadvantage	Floating point - slow
Speed	Moderate
Accuracy	Good but rounding errors possible

1.8.2 Bresenham's Line Algorithm

Bresenham's Line Algorithm DDA से तेज है क्योंकि यह केवल integer arithmetic (addition/subtraction) का उपयोग करता है। यह decide करता है कि अगला pixel ऊपर जाएगा या सीधा।

```
Decision Parameter (pk):

pk = 2*dy - dx  (initial value)

If pk < 0:  Next pixel = (x+1, y)    [right]
            pk_new = pk + 2*dy

If pk >= 0: Next pixel = (x+1, y+1)  [right-up]
            pk_new = pk + 2*dy - 2*dx
```



```
// Bresenham's Line Algorithm
void Bresenham(int x1, int y1, int x2, int y2) {
    int dx = x2 - x1;
    int dy = y2 - y1;
    int pk = 2*dy - dx;  // Initial decision parameter

    int x = x1, y = y1;
    putpixel(x, y, WHITE);

    for (int i = 0; i < dx; i++) {
        x++;
        if (pk < 0) {
            pk = pk + 2*dy;     // y same रहेगा
        } else {
            y++;
            pk = pk + 2*dy - 2*dx;  // y बढ़ेगा
        }
        putpixel(x, y, WHITE);
    }
}

// Example: Line from (0,0) to (5,3)
// dx=5, dy=3, pk0 = 2*3-5 = 1
// Step1: pk=1>=0, y=1, pk=1+6-10=-3, plot(1,1)
// Step2: pk=-3<0,     pk=-3+6=3,     plot(2,1)
// Step3: pk=3>=0, y=2, pk=3+6-10=-1, plot(3,2)
// ...and so on
```

Algorithm	DDA	Bresenham
Arithmetic	Floating point	Integer only
Speed	Slower	Faster

Accuracy	Good	Better
Hardware	Complex	Simple
Round-off errors	Yes	No

NIT - II: Circle Algorithms और Filled-Area Primitives

2.1 Circle Drawing - Midpoint Circle Algorithm

Circle draw करना line से complex है क्योंकि हमें $x^2 + y^2 = r^2$ equation follow करनी होती है। Midpoint Circle Algorithm (Bresenham's Circle Algorithm) efficient integer arithmetic use करता है।

```
Circle Symmetry (8-way):

          (-x,y) | (x,y)
    (-y,x)  -----|-----  (y,x)
            \    |    /
     --------+--+--+--------
            /    |    \
    (-y,-x) ----|----  (y,-x)
         (-x,-y)| (x,-y)
```

एक octant calculate करो, बाकी 7 symmetry से मिलेंगे

```
// Midpoint Circle Algorithm
void MidpointCircle(int cx, int cy, int r) {
    int x = 0;
    int y = r;
    int p = 1 - r;  // Initial decision parameter

    plotCirclePoints(cx, cy, x, y);  // First point

    while (x < y) {
        x++;
        if (p < 0) {
            p = p + 2*x + 1;      // y same
        } else {
            y--;
            p = p + 2*x - 2*y + 1; // y घटेगा
        }
        plotCirclePoints(cx, cy, x, y);
    }
}

// 8-point symmetry से plot करना
void plotCirclePoints(int cx, int cy, int x, int y) {
    putpixel(cx+x, cy+y);  putpixel(cx-x, cy+y);
    putpixel(cx+x, cy-y);  putpixel(cx-x, cy-y);
    putpixel(cx+y, cy+x);  putpixel(cx-y, cy+x);
    putpixel(cx+y, cy-x);  putpixel(cx-y, cy-x);
}

// Example: Circle center(0,0) radius=5
// p0 = 1-5 = -4
// x=0,y=5 -> x=1,y=5 (p<0) -> x=2,y=4 (p>=0) ...
```

2.2 Filled-Area Primitives (भरे हुए क्षेत्र)

Graphics में shapes को color से भरना जरूरी होता है। इसके लिए विभिन्न fill algorithms का उपयोग होता है।

2.2.1 Scan-Line Polygon Fill Algorithm

Scan-line algorithm horizontal scan lines का उपयोग करके polygon भरता है। हर scan line के लिए polygon के edges के साथ intersections find करता है।

```
Polygon:          Scan Lines:
     *            y=6:  |-------|
    / \           y=5:  |---------|
   /   \          y=4:  |-----------|
  *     *          y=3:  |---------|
   \   /           y=2:  |-------|
    \ /            y=1:  |-----|
     *
```

```
Steps:
1. हर scan line के लिए edges के intersection find करो
2. Intersections को sort करो
3. Pairs में pixels fill करो
```

```
// Scan-Line Fill (pseudocode)
void ScanLineFill(Polygon poly) {
    // 1. Find ymin और ymax
    int ymin = poly.getMinY();
    int ymax = poly.getMaxY();

    // 2. हर scan line process करो
    for (int y = ymin; y <= ymax; y++) {
        // 3. Edges के साथ intersections find करो
        List<int> intersections = findIntersections(poly, y);

        // 4. Sort करो
        sort(intersections);

        // 5. Pairs में fill करो
        for (int i = 0; i < intersections.size(); i += 2) {
            fillPixels(intersections[i], intersections[i+1], y);
        }
    }
}
```

2.2.2 Inside-Outside Tests (अंदर-बाहर जांच)

यह test करता है कि कोई point polygon के अंदर है या बाहर।

```
Even-Odd Rule (Parity Test):

Point P से कोई direction में ray निकालो
Polygon boundaries की संख्या count करो

Odd Count  = Point INSIDE  polygon
Even Count = Point OUTSIDE polygon
```

```
   *---*
  /     \
 /   P   \-------> ray crosses 1 time = ODD = INSIDE
*         *
 \       /
  *-----*
```

```
// Even-Odd Rule
bool isInsidePolygon(Point P, Polygon poly) {
    int count = 0;
```

```
    // P से right direction में ray निकालो
    for each edge in poly {
        if (ray intersects edge) {
            count++;
        }
    }
    return (count % 2 == 1); // Odd = inside
}
```

2.2.3 Boundary Fill Algorithm

Boundary Fill एक seed point से शुरू होकर boundary color तक flood fill करता है।

```
  Boundary Fill:
  +--boundary--+
  |            |
  |  * seed    |  <- seed point से शुरू
  |            |
  +------------+

  4-connected: UP, DOWN, LEFT, RIGHT
  8-connected: ऊपर के 4 + 4 diagonals
```

```
// Boundary Fill Algorithm (Recursive, 4-connected)
void boundaryFill(int x, int y, int fillColor, int boundaryColor) {
    int currentColor = getPixel(x, y);

    if (currentColor != boundaryColor &&
        currentColor != fillColor) {

        putpixel(x, y, fillColor);  // current pixel fill करो

        // 4 directions में recursive call
        boundaryFill(x+1, y,   fillColor, boundaryColor); // RIGHT
        boundaryFill(x-1, y,   fillColor, boundaryColor); // LEFT
        boundaryFill(x,   y+1, fillColor, boundaryColor); // UP
        boundaryFill(x,   y-1, fillColor, boundaryColor); // DOWN
    }
}

// Call: boundaryFill(seedX, seedY, RED, WHITE);
```

2.2.4 Flood Fill Algorithm

Flood Fill एक particular color वाले region को दूसरे color से replace करता है।

```
// Flood Fill Algorithm
void floodFill(int x, int y, int newColor, int oldColor) {
    if (getPixel(x, y) == oldColor) {
        putpixel(x, y, newColor);

        floodFill(x+1, y,   newColor, oldColor);
        floodFill(x-1, y,   newColor, oldColor);
        floodFill(x,   y+1, newColor, oldColor);
        floodFill(x,   y-1, newColor, oldColor);
    }
}
```

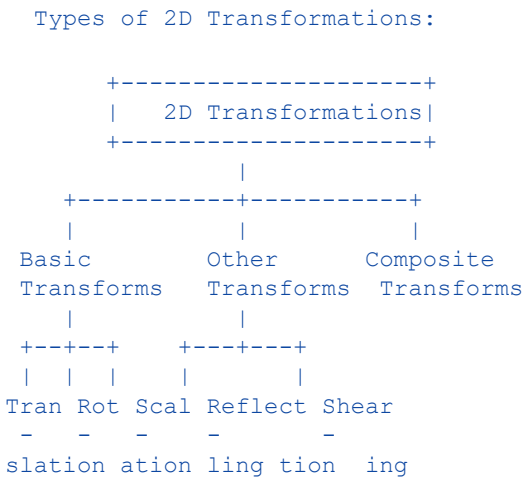
Algorithm	आधार	उपयोग	Limitation
Scan-Line Fill	Horizontal lines	Polygons	Complex edges

Boundary Fill	Boundary color check	Any closed region	Stack overflow for large areas
Flood Fill	Old color replace	Paint bucket tool	Stack overflow

UNIT - III: 2D Geometric Transformations (द्विआयामी ज्यामितीय रूपांतरण)

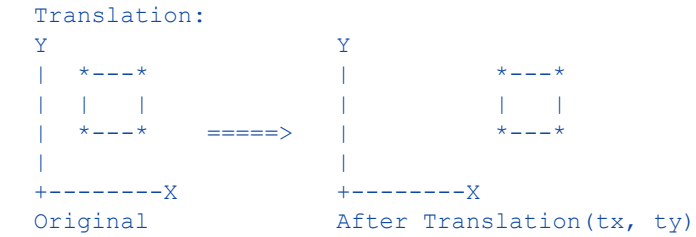
3.1 Transformations का परिचय

2D Transformations किसी object की position, size या shape बदलने की mathematical operations हैं। इनमें Translation, Rotation और Scaling मुख्य हैं।



3.2 Translation (स्थानांतरण)

Translation में object को एक position से दूसरी position पर move किया जाता है।



Formula: x' = x + tx
y' = y + ty

```
// Translation Matrix
| x' |   | 1  0  tx | | x |
| y' | = | 0  1  ty | | y |
| 1  |   | 0  0   1 | | 1 |

// C code for Translation
void translate(Point p, int tx, int ty) {
    int newX = p.x + tx;
    int newY = p.y + ty;
    putpixel(newX, newY, WHITE);
}

// Example: Point (3,4) को (2,3) translate करो
```

```
// x' = 3+2 = 5
// y' = 4+3 = 7
// New Point: (5,7)
```

3.3 Rotation (घुमाव)

Rotation में object को किसी point (usually origin) के चारों ओर θ angle पर rotate किया जाता है।

```
Rotation by angle θ:

Y              Y
|  *           |    *
| /|           |   /|
|/ |    =>     |  / |
+--*---X       +-*--+--X
P(x,y)         P'(x',y')

Formula:
x' = x*cos(θ) - y*sin(θ)
y' = x*sin(θ) + y*cos(θ)
```

```
// Rotation Matrix (Counter-clockwise, θ angle)
| x' |   | cosθ  -sinθ  0 | | x |
| y' | = | sinθ   cosθ  0 | | y |
| 1  |   |  0      0    1 | | 1 |

// C code for Rotation
#include <math.h>
void rotate(Point p, float theta) {
    float rad = theta * 3.14159 / 180; // degrees to radians
    int newX = p.x*cos(rad) - p.y*sin(rad);
    int newY = p.x*sin(rad) + p.y*cos(rad);
    putpixel(newX, newY, WHITE);
}

// Example: Point (1,0) को 90° rotate करो
// x' = 1*cos90 - 0*sin90 = 0
// y' = 1*sin90 + 0*cos90 = 1
// New Point: (0,1) ✓
```

3.4 Scaling (आकार बदलना)

Scaling में object का size बदला जाता है। Sx और Sy scaling factors हैं।

```
Scaling:
Y                Y
|  *--*          |  *-----*
|  |  |   =>     |  |     |
|  *--*          |  *-----*
+------X         +---------X
Original         After Scale(2,2) - 2x size

Formula: x' = x * Sx
         y' = y * Sy

Sx=Sy=1: No change
Sx=Sy>1: Enlarge (बड़ा)
Sx=Sy<1: Shrink (छोटा)
Sx≠Sy:   Non-uniform scaling
```

```
// Scaling Matrix
| x' |   | Sx  0   0 | | x |
```

```
| y' | = |  0  Sy  0 | | y |
| 1  |   |  0  0   1 | | 1 |

// C code for Scaling
void scale(Point p, float sx, float sy) {
    int newX = p.x * sx;
    int newY = p.y * sy;
    putpixel(newX, newY, WHITE);
}

// Example: Point (3,2), Scale by (2,3)
// x' = 3*2 = 6
// y' = 2*3 = 6
// New Point: (6,6)
```

3.5 Homogeneous Coordinates (समजातीय निर्देशांक)

Homogeneous Coordinates में 2D point (x,y) को 3D में (x,y,1) से represent किया जाता है। इससे सभी transformations को एक matrix multiplication से represent कर सकते हैं।

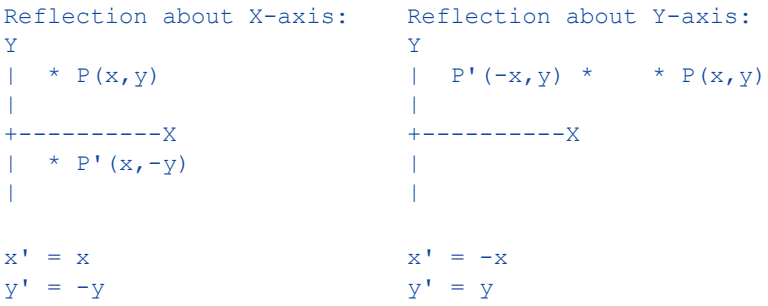
Transformation	Matrix	Formula
Translation	[[1,0,tx],[0,1,ty],[0,0,1]]	x'=x+tx, y'=y+ty
Rotation	[[cosθ,-sinθ,0],[sinθ,cosθ,0],[0,0,1]]	Rotate by θ
Scaling	[[Sx,0,0],[0,Sy,0],[0,0,1]]	x'=x*Sx, y'=y*Sy
Reflection X	[[1,0,0],[0,-1,0],[0,0,1]]	y'=-y
Reflection Y	[[-1,0,0],[0,1,0],[0,0,1]]	x'=-x
Shearing	[[1,Shx,0],[Shy,1,0],[0,0,1]]	x'=x+Shx*y

3.6 Composite Transformations (संयुक्त रूपांतरण)

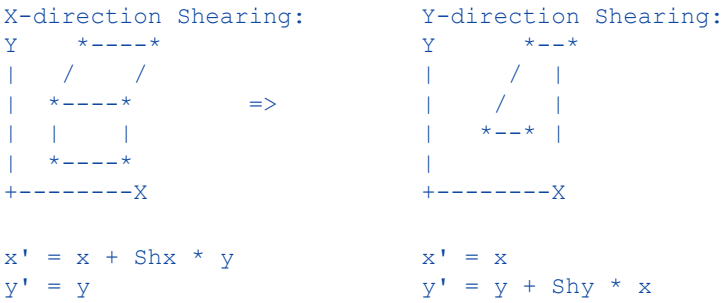
जब multiple transformations एक साथ apply की जाती हैं तो उनकी matrices को multiply करके composite matrix बनाते हैं।

```
// Composite Transformation Example:
// Scale then Rotate then Translate

// Individual Matrices:
T = Translation Matrix
R = Rotation Matrix
S = Scaling Matrix

// Composite Matrix:
M = T * R * S  (right to left apply होता है)

// Point transformation:
P' = M * P

// Note: Matrix multiplication commutative नहीं है
// T*R ≠ R*T  (order matters!)
```

3.7 Reflection (प्रतिबिम्ब)



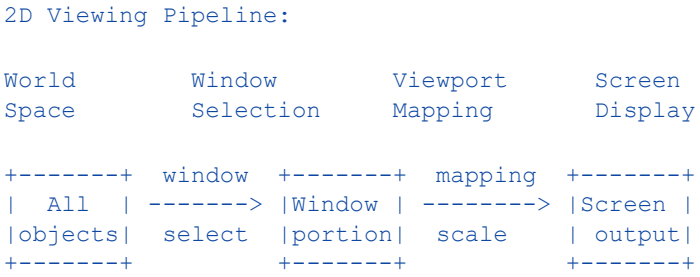
3.8 Shearing (तिरछा करना)



UNIT - IV: 2D Viewing और Clipping

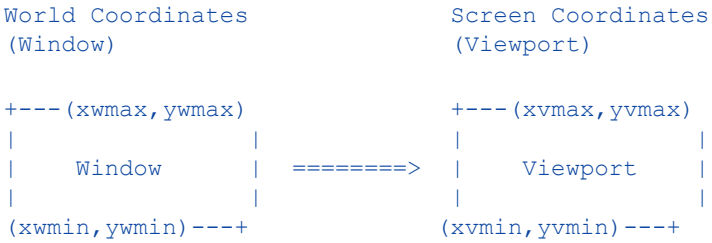
4.1 Viewing Pipeline (व्यूइंग पाइपलाइन)

2D Viewing Pipeline वह process है जिसके द्वारा world coordinates से screen coordinates में conversion होती है।



Step 1: World Space में objects define करो
Step 2: Window (क्या दिखाना है) select करो
Step 3: Clipping - window के बाहर remove करो
Step 4: Viewport mapping - screen पर map करो

4.2 Window to Viewport Transformation



```
Mapping Formula:
xv = xvmin + (xw-xwmin) * (xvmax-xvmin)/(xwmax-xwmin)
yv = yvmin + (yw-ywmin) * (yvmax-yvmin)/(ywmax-ywmin)
```

4.3 Point Clipping (बिंदु क्लिपिंग)

Point Clipping check करती है कि कोई point window के अंदर है या बाहर।

```
// Point Clipping
bool pointClip(Point P, Window W) {
    if (P.x >= W.xmin && P.x <= W.xmax &&
        P.y >= W.ymin && P.y <= W.ymax) {
        return true;   // INSIDE - display करो
    }
    return false;      // OUTSIDE - discard करो
}
```

4.4 Cohen-Sutherland Line Clipping Algorithm

Cohen-Sutherland Algorithm lines को clip करने का सबसे popular algorithm है। यह 4-bit outcode system use करता है।

```
Region Codes (Outcodes):

        |          |
1001    |  0001    |  0000  <- window के अंदर
(TL)    |  (T)     |  (TR)
        |          |
--------+----------+-------  <- window top edge
        |          |
1000    |  0000    |  0010
(L)     | (INSIDE) |  (R)
        |          |
--------+----------+-------  <- window bottom edge
        |          |
0110    |  0100    |  0110
(BL)    |  (B)     |  (BR)
        |          |

Bit: TOP  BOTTOM  RIGHT  LEFT
     bit3  bit2   bit1   bit0
```

```
// Cohen-Sutherland Outcodes
#define LEFT   1  // 0001
#define RIGHT  2  // 0010
#define BOTTOM 4  // 0100
#define TOP    8  // 1000

int computeCode(float x, float y, Window w) {
    int code = 0;
    if (x < w.xmin) code |= LEFT;
    if (x > w.xmax) code |= RIGHT;
    if (y < w.ymin) code |= BOTTOM;
    if (y > w.ymax) code |= TOP;
    return code;
}

void CohenSutherland(Point P1, Point P2, Window w) {
    int code1 = computeCode(P1.x, P1.y, w);
    int code2 = computeCode(P2.x, P2.y, w);
    bool accept = false;
```

```
    while (true) {
        if (!(code1 | code2)) {          // दोनों अंदर
            accept = true; break;
        } else if (code1 & code2) {      // दोनों बाहर (same side)
            break;  // Reject
        } else {
            // Intersection calculate करो
            int codeOut = code1 ? code1 : code2;
            float x, y;
            if (codeOut & TOP)
                x = P1.x + (P2.x-P1.x)*(w.ymax-P1.y)/(P2.y-P1.y);
            // ... other edges similarly
        }
    }
    if (accept) drawLine(P1, P2);
}
```

Cohen-Sutherland Cases:

Case	Condition	Action
Trivially Accept	code1=0 AND code2=0	Line completely inside - draw करो
Trivially Reject	code1 AND code2 ≠ 0	Line completely outside - discard
Partial Clip	दोनों cases नहीं	Intersection find करो, clip करो

4.5 Sutherland-Hodgeman Polygon Clipping

यह algorithm polygon को clip करता है। यह polygon के हर edge को हर clipping boundary के साथ process करता है।

```
Steps:
1. Start with original polygon vertices
2. हर clipping edge (LEFT, RIGHT, BOTTOM, TOP) के लिए:
   a. हर edge के दोनों endpoints check करो
   b. 4 cases handle करो:

 Case 1: Both INSIDE -> P2 output में add करो
 Case 2: P1 inside, P2 outside -> Intersection add करो
 Case 3: Both OUTSIDE -> कुछ नहीं
 Case 4: P1 outside, P2 inside -> Intersection और P2 add करो
```

```
+------+          +---+----+
|  *--*|          |   |clip|
| /    |  Clip    | *-+*  |
|*     |  =====>  ||  |   |
| \    |          | *-+*  |
|  *--*|          |   | clip|
+------+          +---+----+
```

4.6 Weiler-Atherton Polygon Clipping

यह algorithm concave polygons और windows दोनों को handle कर सकता है, जो Sutherland-Hodgeman नहीं कर सकता।

```
Weiler-Atherton Features:
```

```
- Concave polygons को handle करता है
- Complex self-intersecting shapes
- Polygon के अंदर holes भी handle करता है

Entering and Exiting Intersections:
+----------+
|  window  |
|    +---> | entering intersection
|    |     |
|    +<--- | exiting intersection
+----------+
```

4.7 Text Clipping

Text को clip करने के तीन methods हैं:

Method	Description	Result
All or None String	पूरा text window में हो तो display	Simple but wastes space
All or None Character	हर character individually check	Better approach
Component Clipping	Individual parts clip	Best result but complex

UNIT - V: 3D Concepts और Transformations

5.1 3D Computer Graphics का परिचय

3D Graphics में objects को तीन dimensions (X, Y, Z) में represent किया जाता है। यह 2D से अधिक complex है लेकिन real world को better represent करता है।

```
3D Coordinate System:

            Z (ऊपर)
            |
            |
            |
            +-----------> X (दाईं ओर)
           /
          /
         v
        Y (आगे की ओर)

Point in 3D: P(x, y, z)
Homogeneous: P(x, y, z, 1)
```

5.2 3D Translation (3D स्थानांतरण)

```
// 3D Translation
// x' = x + tx
// y' = y + ty
// z' = z + tz

// 4x4 Translation Matrix:
| x' |   | 1  0  0  tx | | x |
| y' |   | 0  1  0  ty | | y |
| z' | = | 0  0  1  tz | | z |
```

```
| 1  |   | 0  0  0   1 | | 1 |

// Example: Point(2,3,4) translate by (1,2,3)
// x' = 2+1 = 3
// y' = 3+2 = 5
// z' = 4+3 = 7
// Result: (3,5,7)
```

5.3 3D Rotation (3D घुमाव)

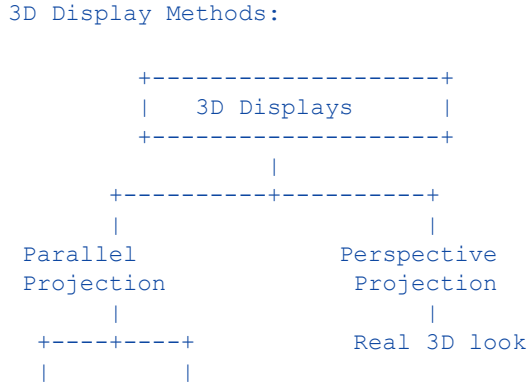
3D Rotation तीन axes के बारे में हो सकती है: X-axis, Y-axis, या Z-axis।

```
// Rotation about Z-axis (θ angle)
| x' |   | cosθ  -sinθ  0   0 | | x |
| y' | = | sinθ   cosθ  0   0 | | y |
| z' |   |  0      0    1   0 | | z |
| 1  |   |  0      0    0   1 | | 1 |

// Rotation about X-axis
| x' |   | 1    0      0    0 | | x |
| y' | = | 0   cosθ  -sinθ  0 | | y |
| z' |   | 0   sinθ   cosθ  0 | | z |
| 1  |   | 0    0      0    1 | | 1 |

// Rotation about Y-axis
| x' |   |  cosθ  0   sinθ  0 | | x |
| y' | = |   0    1    0    0 | | y |
| z' |   | -sinθ  0   cosθ  0 | | z |
| 1  |   |   0    0    0    1 | | 1 |
```

5.4 3D Scaling (3D आकार बदलना)

```
// 3D Scaling
// x' = x * Sx
// y' = y * Sy
// z' = z * Sz

// 4x4 Scaling Matrix:
| x' |   | Sx  0   0   0 | | x |
| y' |   |  0  Sy  0   0 | | y |
| z' | = |  0  0   Sz  0 | | z |
| 1  |   |  0  0   0   1 | | 1 |
```

5.5 3D Display Methods (3D प्रदर्शन विधियाँ)



```
Orthographic  Oblique
   |
 Top/Front/
 Side views
```

5.6 Parallel Projection (समानांतर प्रक्षेपण)

Parallel Projection में projection lines एक दूसरे के parallel होती हैं। इसमें object का size distance के साथ नहीं बदलता।

```
Parallel Projection:

3D Object           2D Projection
    *-----*              *-----*
   /|    /|      =>      |     |
  * |   * |              |     |
  | *--|-*               *-----*
  |/   |/
  *----*


Projection lines: सभी parallel
Size: same regardless of distance
Use: Engineering, CAD drawings

Types:
- Orthographic (90° angle) - Top, Front, Side view
- Oblique (45° angle) - Cabinet, Cavalier
```

```
// Orthographic Projection (onto XY plane)
// Simply Z-coordinate drop करो
x' = x
y' = y
z' = 0  (ignore z)

// Matrix form:
| x' |   | 1  0  0  0 | | x |
| y' |   | 0  1  0  0 | | y |
| z' | = | 0  0  0  0 | | z |
| 1  |   | 0  0  0  1 | | 1 |
```

5.7 Perspective Projection (परिप्रेक्ष्य प्रक्षेपण)

Perspective Projection real human vision जैसी है जहाँ दूर की objects छोटी दिखती हैं। यह depth का illusion देती है।

```
Perspective Projection:

Eye/Camera              Screen
    *                  *-----*
     \                /       \
      \              /         \
       \    *-------*           *
        \  /  Object             \
         \/                       *-----*
                                 (projected)


Closer = Larger on screen
Farther = Smaller on screen (realistic!)

Formula: x' = x * d/z
         y' = y * d/z
 (d = distance from eye to projection plane)
```

```
// Perspective Projection
// d = distance from Center of Projection
```

```
float d = 500;  // projection plane distance

float perspX = (x * d) / z;
float perspY = (y * d) / z;

// Matrix form (Perspective):
| x' |   | 1  0  0     0  | | x |
| y' |   | 0  1  0     0  | | y |
| z' | = | 0  0  1     0  | | z |
| w' |   | 0  0  1/d   0  | | 1 |

// Final: x_screen = x'/w'
//        y_screen = y'/w'
```

विशेषता	Parallel Projection	Perspective Projection
Projection lines	Parallel	Meet at one point (COP)
Distance effect	Size same रहता है	दूर = छोटा
Realism	कम realistic	अधिक realistic
Use	Engineering, CAD	Games, Animation, Films
Calculation	Simple	Complex
Depth	No depth cue	Strong depth cue

5.8 3D Transformations - Complete Summary

```
3D Transformation Summary:

+-------------------------------------------------------------+
| Transform  |  Matrix Size  |  Key Parameters                |
|------------|---------------|--------------------------------|
| Translation|    4x4        |  tx, ty, tz                    |
| Rotation-X |    4x4        |  θ (angle about X axis)        |
| Rotation-Y |    4x4        |  θ (angle about Y axis)        |
| Rotation-Z |    4x4        |  θ (angle about Z axis)        |
| Scaling    |    4x4        |  Sx, Sy, Sz                    |
| Composite  |    4x4        |  M = T*R*S (order matters)     |
+-------------------------------------------------------------+
```